PARAKH — Stage 2 PRD

Evidentiary Confidence Engine (Confidence Layer)

1. Objective

Build a **deterministic, mathematically grounded confidence scoring system** that assigns each record a score:

$$
C(r) \in [0,1]
$$

This score represents **how trustworthy the record is**, independent of whether it is anomalous.

The system must:

- Quantify **data reliability**
- Penalize **missing, inconsistent, and contradictory data**
- Produce interpretable sub-components:
    - completeness
    - temporal coherence
    - reconciliation

2. Scope

Included

- Computation of:
    - C_{comp}
    - C_{temp}
    - C_{recon}
- Log-space aggregation into final confidence
- Per-record confidence breakdown
- Confidence reports and summaries

Excluded

- Risk scoring
- ML models
- Routing decisions

- External data sources

---

3. Inputs

From Stage 1:

```
Corpus.records
```

Each record must already be:

- schema-validated
- normalized
- cleaned

---

4. Output

Each record must produce:

```
{
  "confidence": float,
  "components": {
    "completeness": float,
    "temporal": float,
    "reconciliation": float
  }
}
```

---

5. Mathematical Specification

5.1 Final Confidence

$$
C(r) = \exp\left(
w_1 \log C_{\text{comp}} +
w_2 \log C_{\text{temp}} +
w_3 \log C_{\text{recon}}
\right)
$$

Constraints:

- $w_1 + w_2 + w_3 = 1$
- Default: $w_1 = w_2 = w_3 = \frac{1}{3}$

5.2 Completeness Score

$$
C_{\text{comp}}(r) =
\frac{\sum_{f \in F} v_f \cdot \mathbb{1}[r_f \text{ valid}]}
{\sum_{f \in F} v_f}
$$

Field weights:

$$
v_f = (1 - H_{\text{null}}(f)) \cdot H_{\text{value}}(f)
$$

Implementation Details

- Compute entropy across entire corpus
- Normalize entropy to [0,1]
- Cache (v_f) for reuse

Validity Conditions

A field is valid if:

- Not null
- Not placeholder
- Correct type

5.3 Temporal Coherence

Let ordered pairs:

```
O = [
  ("date_proposal", "date_approval"),
  ("date_approval", "date_completion")
]
```

$$
C_{\text{temp}}(r) =
\prod_{(a,b) \in O}
\begin{cases}
1 & t_b \ge t_a \\
\exp(-\kappa |t_b - t_a|) & t_b < t_a

\end{cases}
$$

Parameters

- (\kappa) default: `0.01`

Hard Fail Conditions

If:

- Any date < 1993 (scheme start)
- Unparseable dates

Then:

$$
C_{\text{temp}} = 0
$$

5.4 Reconciliation Score

Compare:

- `sanction_amount`
- `amount_spent`

$$
C_{\text{recon}}(r) =
\exp\left(
-\lambda \cdot
\frac{|x_1 - x_2|}
{|x_1| + |x_2| + \epsilon}
\right)
$$

Parameters

- (\lambda = 2.0)
- (\epsilon = 1e-6)

Edge Cases

- Both values null → ignore component (set = 1)
- One null → penalize (set low, e.g. 0.2)

6. Implementation Architecture

6.1 Module Structure

- `confidence.py`
- `entropy.py`
- `validators.py`

6.2 Core Class

```
class ConfidenceModel:
    def __init__(self, weights=None):
        self.weights = weights or (1/3, 1/3, 1/3)

    def score(self, corpus: Corpus):
        return ConfidenceResult
```

6.3 Output Object

```
class ConfidenceResult:
    scores: List[float]
    breakdown: List[Dict]
```

7. Non-Functional Requirements

Performance

- 50k records processed in < 3 seconds

Determinism

- Same input → same output

Numerical Stability

- All operations in log-space

8. Validation & Testing

8.1 Unit Tests

Test cases:

- All fields valid → high confidence (>0.9)
- Missing fields → low completeness
- Invalid dates → temporal = 0
- Mismatched amounts → low reconciliation

8.2 Sanity Checks

Run:

```
print(confidence.mean())
print(confidence.min(), confidence.max())
```

Expected:

- Range within [0,1]
- Distribution not collapsed

8.3 Synthetic Validation

Use Stage 1 generated noise:

- Missing data → lower C
- Date violations → near 0
- Clean records → near 1

---

9. Edge Cases (MANDATORY)

Handle:

- All fields missing
- All dates invalid
- Extremely large values
- Identical values (no variance)

---

10. Output Reports

10.1 Summary

```
{
  "mean_confidence": 0.62,
  "low_confidence_pct": 28.4,
```

```
  "high_confidence_pct": 35.1
}
```

10.2 Distribution

- Histogram of confidence scores
- Breakdown of components

---

11. Acceptance Criteria

Stage complete if:

- ☐ Confidence computed for all records
- ☐ Component scores available
- ☐ Edge cases handled correctly
- ☐ Outputs are stable and interpretable
- ☐ Works on synthetic dataset

---

12. Definition of Done

- Confidence pipeline runs end-to-end
- Produces meaningful differentiation between good and bad data
- Ready for Stage 3 (Clustering & Peer Formation)

---

13. Key Design Principle

> **Do not guess truth. Quantify uncertainty.**

This layer does NOT detect fraud.
It determines whether the data is even worth trusting.
